INSTITUTO FEDERAL DE EDUCACAO, CIENCIA E TECNOLOGIA - CAMPUS PICOS
Programacao Orientada a Objetos - 2026.2
Pratica POO 02 - Encapsulamento
Prof. Joao Paulo

Respostas - Praticando conceitos avancados

1. getTitular() e encapsulamento

`contaDoCarlos.getTitular().setNome("Nome Alterado")` **altera sim** o estado interno do `Aluno`, pois `getTitular()` devolve a referência real ao objeto, não uma cópia. Como `Aluno` expõe `setNome()`, qualquer código com acesso à conta pode alterar o aluno por trás da `ContaEstudantil`, embora ela só quisesse expor leitura. Isso **viola o encapsulamento**, porque o controle sobre a mutação do `Aluno` deixa de ser exclusivo da própria classe `Aluno`.

Abordagem mais segura: não permitir alteração de `nome`/`email` por quem não é o próprio `Aluno` (removendo os setters públicos e expondo apenas métodos de domínio específicos, se necessário), ou fazer `getTitular()` devolver uma visão somente leitura do aluno (uma interface ou DTO sem setters), em vez do objeto mutável original.

2. Envio de e-mail dentro de pagar()

Sim, fere o **Single Responsibility Principle**: `ContaEstudantil` passaria a ter duas razões para mudar, a regra financeira do pagamento e a lógica de notificação (servidor SMTP, formato do e-mail, etc.). Misturar as duas torna a classe mais difícil de testar e de evoluir (trocar o provedor de e-mail não deveria exigir mexer na regra de pagamento).

O código de envio deveria ficar em uma classe própria, dedicada à notificação (ex.: um serviço de notificação), acionada por quem orquestra a operação após `pagar()` retornar `true`, e não pela própria entidade de domínio.

3. Acoplamento em transferir(ContaEstudantil destino, ...)

a) O acoplamento entre `ContaEstudantil` e ela mesma é **aceitável**: é a própria classe operando sobre outra instância do mesmo tipo, dentro do mesmo domínio, através do seu próprio contrato público (`pagar`/`depositar`). Não há dependência de algo externo ao domínio.

b) Se `transferir` recebesse diretamente uma classe concreta externa, como `BancoDoBrasilAPI`, o acoplamento aumentaria bastante: `ContaEstudantil` passaria a depender de uma implementação específica de um sistema externo, ficando vulnerável a qualquer mudança nessa API e mais difícil de testar isoladamente. O encapsulamento resolve isso escondendo os detalhes da validação externa atrás de uma **interface** (ex.: um validador de transferências), que `ContaEstudantil` conhece apenas pelo contrato. A implementação concreta (`BancoDoBrasilAPI` ou outra) é injetada de fora, podendo ser trocada sem alterar a classe de domínio, aplicando o princípio da inversão de dependência.